CS 466 / 666 Information Retrieval
Final Project Report

Hybrid Retrieval Fusion: A Systematic Comparison of Five Fusion Strategies

Team members	Jian Huang, Qingchen Li
Section(s)	666
Email(s)	jhuan243@jh.edu, qli137@jh.edu
External repository	https://github.com/jayhuang298-ops/cs466-hybrid-retrieval-fusion

Project summary: This project studies how to combine sparse lexical retrieval and dense semantic retrieval for robust information retrieval across domains. We compare BM25 and BGE-base-en-v1.5 baselines with five hybrid fusion strategies: linear score interpolation, reciprocal rank fusion, convex rank fusion, learned logistic-regression fusion, and query-adaptive random-forest fusion. The system is evaluated on BEIR datasets with NDCG@10, MAP, Recall@100, paired significance tests, hyperparameter sensitivity analysis, query-level analysis, and an oracle upper-bound study.

April 2026

Contents

1 Introduction

Modern information retrieval systems often need to match different types of evidence. Some queries depend heavily on lexical matching, such as named entities, technical terminology, or exact phrases. In other cases, semantic matching is needed because a relevant document may express the same idea as the query while using different words. Sparse retrieval algorithms such as BM25 perform well in the former setting, while neural dense retrieval algorithms are designed for the latter. However, neither approach consistently outperforms the other in every situation.

In this project, we investigate how to combine both approaches. BM25 is used as the lexical sparse retriever, while BGE-base-en-v1.5 is used as the semantic dense retriever. The goal is to compare multiple techniques for fusing their outputs into a unified hybrid retrieval system. Instead of treating hybrid retrieval as a single method, we analyze it as a design space: score-based, rank-based, supervised, and query-adaptive approaches each make different assumptions about the reliability of the base retrievers' outputs.

The key problem investigated in this project is: when fusing BM25 and BGE dense retrieval, which fusion techniques work best across different retrieval domains, and how much potential for further improvement remains in query-adaptive fusion?

To answer this question, we developed an evaluation pipeline based on the BEIR evaluation paradigm. We use MS MARCO for training and tuning, while using TREC-COVID, ArguAna, and FiQA for cross-domain evaluation. In addition to measuring retrieval effectiveness, we also study the behavior of the fusion methods in more detail through hyperparameter sensitivity analysis, statistical significance testing, query-level analysis, and an oracle upper-bound study.

This report is organized as follows. Section 2 explains how to run the submitted code and reproduce the main outputs. Section 3 describes the system architecture and the retrieval models involved. Section 4 discusses the achievements of our project. Section 5 presents the evaluation process and experimental results. Sections 6 and 7 describe the expected outputs, limitations, and future extensions. Section 8 discusses external dependencies and AI assistance.

2 User Guide: How to Run the Code

2.1 Repository

The complete source code is available at:

https://github.com/jayhuang298-ops/cs466-hybrid-retrieval-fusion

The repository contains configuration files, scripts, source modules, trained fusion models, notebooks, generated tables, and generated figures. The large raw data, indexes, and run files are intentionally excluded from Git because they can be regenerated and are too large for normal repository storage.

2.2 Expected Environment

The project was designed for Python with common information retrieval and machine learning libraries. The important dependencies include:

- `beir` for loading benchmark datasets,
- `pyserini` and Java/Lucene for BM25 retrieval,

- `faiss-cpu` or `faiss-gpu` for dense nearest-neighbor search,
- `torch`, `transformers`, and `sentence-transformers` for BGE embeddings,
- `pytrec_eval` for IR metric computation,
- `scikit-learn` and `scipy` for supervised fusion and paired significance testing,
- `matplotlib`, `numpy`, and related utilities for analysis and reporting.

2.3 Quickstart

A typical run is:

```
bash scripts/01_setup_env.sh
conda activate hybridir

python scripts/02_download_data.py

# Generate BM25 and dense runs.
# The repository includes a Colab notebook for GPU-based encoding:
# notebooks/colab_compute.ipynb

python scripts/06_train_fusion_models.py --strategy all
python scripts/08_run_experiments.py
python scripts/09_generate_figures.py
python scripts/10_generate_tables.py
python scripts/11_write_analysis.py
python scripts/12_generate_description_tables.py
```

For large dense-encoding jobs, the project uses Google Colab with a T4 GPU. Local CPU or Apple M-series hardware is sufficient for fusion, evaluation, and table/figure generation after the retrieval runs have been generated.

3 System Design and Implementation

3.1 Architecture

The codebase is organized around a separation between data loading, retrieval, fusion, evaluation, and reporting:

```
configs/          YAML configuration files
src/data/         BEIR data loading and feature extraction
src/retrieval/    BM25 and dense retrieval modules
src/fusion/       Fusion strategies A-E
src/eval/         Metric computation and significance testing
src/utils/        Run-file I/O, normalization, logging, config helpers
scripts/          End-to-end experiment and reporting scripts
notebooks/        Colab notebook for GPU encoding
models/trained/   Learned fusion model weights
report/           Generated tables, figures, and findings
```

The central design principle is *retrieve once, fuse many times*. BM25 and dense retrieval are expensive, especially when dense corpus embeddings must be built. After the top-1000 BM25 and

dense results are cached, all fusion strategies can be run quickly and consistently over the same candidate sets.

3.2 Datasets

MS MARCO is used for training and tuning. Three BEIR datasets are used for zero-shot evaluation: TREC-COVID, ArguAna, and FiQA. This setup tests whether fusion strategies tuned on a general web question-answering dataset transfer to biomedical, argumentative, and financial domains.

Table 1: Dataset statistics. MS MARCO is sub-sampled to 500K documents via reservoir sampling for computational feasibility and is used only for training and tuning, not zero-shot evaluation.

Dataset	Domain	Split	Role	#Corpus	#Queries	#Qrels	Relevance
MS MARCO	Web / Q&A	dev	Train & Tune	500K	6,980	7,437	Binary
TREC-COVID	Biomedical	test	Eval	171,332	50	66,336	Graded
ArguAna	Argument / Debate	test	Eval	8,674	1,406	1,406	Binary
FiQA	Financial Q&A	test	Eval	57,638	648	1,706	Binary

3.3 Fusion Strategies

The project evaluates BM25 and Dense baselines plus five fusion methods. Table 2 summarizes the methods.

Table 2: Retrieval models and fusion strategies evaluated. Supervised strategies are trained on MS MARCO and evaluated zero-shot on TREC-COVID, ArguAna, and FiQA.

ID	Strategy	Type	Tuned Parameter	Description
–	BM25	Baseline	–	Sparse lexical retrieval using Pyserini/Lucene BM25 scores.
–	Dense BGE	Baseline	–	Semantic retrieval using `BAAI/bge-base-en-v1.5` embeddings and FAISS.
A	Linear Interpolation	Unsupervised	α	Per-query min-max normalized dense and BM25 scores are linearly combined.
B	Reciprocal Rank Fusion	Unsupervised	k	Rank-only fusion using reciprocal rank contributions from each retriever.
C	Convex Rank Fusion	Unsupervised	β	Rank scores are normalized and interpolated between dense and BM25 rankings.
D	Learned Fusion	Supervised	$\mathbf{w}$	Logistic regression over query-document features such as raw scores, normalized scores, ranks, and membership indicators.
E	Query-Adaptive Fusion	Supervised	θ	A random forest predicts a query-specific interpolation weight $\hat{\alpha}(q)$ from query features.

3.3.1 Strategy A: Linear Score Interpolation

Strategy A combines normalized scores:

$$s_A(q,d) = (1-\alpha)\,\tilde{s}_{\text{dense}}(q,d) + \alpha\,\tilde{s}_{\text{BM25}}(q,d). \tag{1}$$

Scores are min-max normalized within each query's candidate pool. Missing documents from one list are assigned zero after normalization. The interpolation weight α is tuned on MS MARCO and then frozen for zero-shot evaluation.

3.3.2 Strategy B: Reciprocal Rank Fusion

Strategy B ignores score magnitudes and uses ranks:

$$s_B(q,d) = \sum_{r\in\{\text{BM25,dense}\}} \frac{1}{k + \text{rank}_r(q,d)}. \tag{2}$$

The method is robust because it does not require comparable score scales, but it may discard useful confidence information from the dense retriever.

3.3.3 Strategy C: Convex Rank Fusion

Strategy C normalizes ranks to $[0,1]$ and then interpolates them:

$$s_C(q,d) = (1-\beta)\,\tilde{r}_{\text{dense}}(q,d) + \beta\,\tilde{r}_{\text{BM25}}(q,d). \tag{3}$$

It can be viewed as a rank-based analogue of Strategy A.

3.3.4 Strategy D: Learned Logistic-Regression Fusion

Strategy D trains a logistic regression classifier on MS MARCO. For each query-document pair in the candidate pool, it uses pairwise features including BM25 score, dense score, normalized scores, normalized ranks, and indicators for whether the document appears in each top-k list. The model outputs a relevance probability and ranks documents by that probability.

3.3.5 Strategy E: Query-Adaptive Fusion

Strategy E tries to learn when to rely more on BM25 or dense retrieval. First, an oracle interpolation weight $\alpha^*(q)$ is computed for each training query by grid search. Then a random forest predicts $\hat{\alpha}(q)$ from query-level features such as query length, stopword ratio, question flag, and BM25 score distribution. At inference time, Strategy E applies Strategy A using the predicted query-specific weight.

4 Project Achievements, Strengths, and Selling Points

1. **Complete hybrid retrieval pipeline.** The project implements data loading, indexing, sparse retrieval, dense retrieval, fusion, evaluation, statistical testing, and automatic figure/table generation.

2. **Systematic comparison of five fusion strategies.** Instead of only testing one hybrid method, the project compares simple unsupervised methods, rank-based methods, supervised document-level fusion, and adaptive query-level fusion.

3. **Cross-domain evaluation.** Training and tuning are performed on MS MARCO, while evaluation is conducted zero-shot on biomedical, argumentative, and financial datasets. This makes the conclusions more meaningful than a single-dataset experiment.

4. **Efficient experimental design.** The retrieve-once, fuse-many-times architecture avoids repeatedly running expensive BM25 indexing and dense encoding. This makes it easy to iterate on fusion methods and evaluation scripts.

5. **Statistical testing.** The project saves per-query metric vectors and uses paired t-tests, allowing performance improvements to be evaluated beyond aggregate scores.

6. **Interpretability.** Hyperparameter sensitivity curves, query-bin analysis, and oracle upper bounds help explain why some fusion methods work and why adaptive fusion remains difficult.

7. **Practical reproducibility.** Scripts are numbered in pipeline order, configuration is separated into YAML files, and generated artifacts are organized under `report/`.

Overall, the project is complex in scope because it combines multiple IR systems, dense neural retrieval, classical ranking, supervised learning, statistical evaluation, and cross-domain benchmark analysis. Its strongest empirical result is that simple score-aware fusion methods consistently improve over single-system retrieval, while the adaptive analysis provides a clear explanation of the remaining headroom.

5 Empirical Evaluation

5.1 Metrics

The project evaluates retrieval quality using standard IR metrics:

- **NDCG@10**: the main metric, emphasizing ranking quality near the top of the result list;
- **MAP**: mean average precision over judged relevant documents;
- **Recall@100**: coverage of relevant documents within the top 100 results;
- **Paired t-tests**: significance tests over per-query NDCG@10 vectors.

5.2 Main Results

Table 3 shows zero-shot performance across the three evaluation datasets.

Table 3: Zero-shot retrieval performance across BEIR datasets. Bold indicates the best value in each dataset/metric column. † indicates significantly better than B-RRF on NDCG@10 with paired t-test, $p < 0.05$.

Strategy	TREC-COVID			ArguAna			FiQA		
	NDCG@10	MAP	R@100	NDCG@10	MAP	R@100	NDCG@10	MAP	R@100
BM25	0.5947	0.1871	0.1091	0.2999	0.2076	0.9324	0.2361	0.1941	0.5395
Dense BGE	0.7807	0.2581	0.1406	0.4558	0.3243	**0.9929**	0.4062	0.3471	0.7415
A – Linear	0.8233†	**0.3111**	**0.1555**	**0.4576**†	0.3241	0.9922	0.4194†	**0.3574**	0.7203
B – RRF	0.7653	0.2936	0.1425	0.4193	0.2911	0.9886	0.3857	0.3193	0.7177
C – Convex Rank	0.7807	0.2807	0.1406	0.4558†	**0.3243**	**0.9929**	0.4062†	0.3472	0.7415
D – Learned	**0.8271**†	0.1485	0.1403	0.4446†	0.3151	0.9822	**0.4194**†	0.3545	**0.7441**
E – Adaptive	0.8152†	0.3095	0.1543	0.4422†	0.3120	0.9886	0.4130†	0.3517	0.7213

The main findings are:

- Fusion is beneficial overall, but the gains depend strongly on the fusion strategy. The best fusion method improves over dense-only retrieval on each evaluation dataset, while weaker fusion methods such as RRF can underperform the dense baseline.
- Strategy D has an important limitation on TREC-COVID. Although it achieves the best NDCG@10, its MAP is much lower than that of the other methods. This suggests that the learned fusion model is effective at placing some highly relevant documents near the top of the ranking, but is less reliable across the broader ranked list, measured by average precision.
- Strategy A is the strongest simple method and is the most consistently competitive across domains.
- Strategy B underperforms the score-aware methods, suggesting that score magnitudes contain useful information that rank-only RRF discards in this setting.

5.3 Significance Testing

Table 4 compares fusion strategies against the B-RRF baseline using paired t-tests on per-query NDCG@10.

Table 4: Paired t-test results comparing each fusion strategy against B-RRF on NDCG@10.

Dataset	Strategy	t	p	n	Significant
TREC-COVID	A – Linear	+3.447	0.0012	50	Yes
TREC-COVID	C – Convex Rank	+0.780	0.4393	50	No
TREC-COVID	D – Learned	+3.869	0.0003	50	Yes
TREC-COVID	E – Adaptive	+3.581	0.0008	50	Yes
ArguAna	A – Linear	+12.247	0.0000	1406	Yes
ArguAna	C – Convex Rank	+9.903	0.0000	1406	Yes
ArguAna	D – Learned	+8.604	0.0000	1406	Yes
ArguAna	E – Adaptive	+9.664	0.0000	1406	Yes
FiQA	A – Linear	+6.632	0.0000	648	Yes
FiQA	C – Convex Rank	+2.864	0.0043	648	Yes
FiQA	D – Learned	+7.420	0.0000	648	Yes
FiQA	E – Adaptive	+4.706	0.0000	648	Yes

Eleven out of twelve comparisons are significant at $p < 0.05$, supporting the conclusion that most score-aware and learned fusion methods provide reliable gains over RRF in this experiment.

5.4 Hyperparameter Sensitivity

Strategy A uses an interpolation parameter α, where $\alpha = 0$ corresponds to dense-only retrieval and $\alpha = 1$ corresponds to BM25-only retrieval. The sensitivity analysis shows that the three zero-shot evaluation datasets peak near $\alpha = 0.05$, while MS MARCO peaks at a slightly larger interpolation weight.

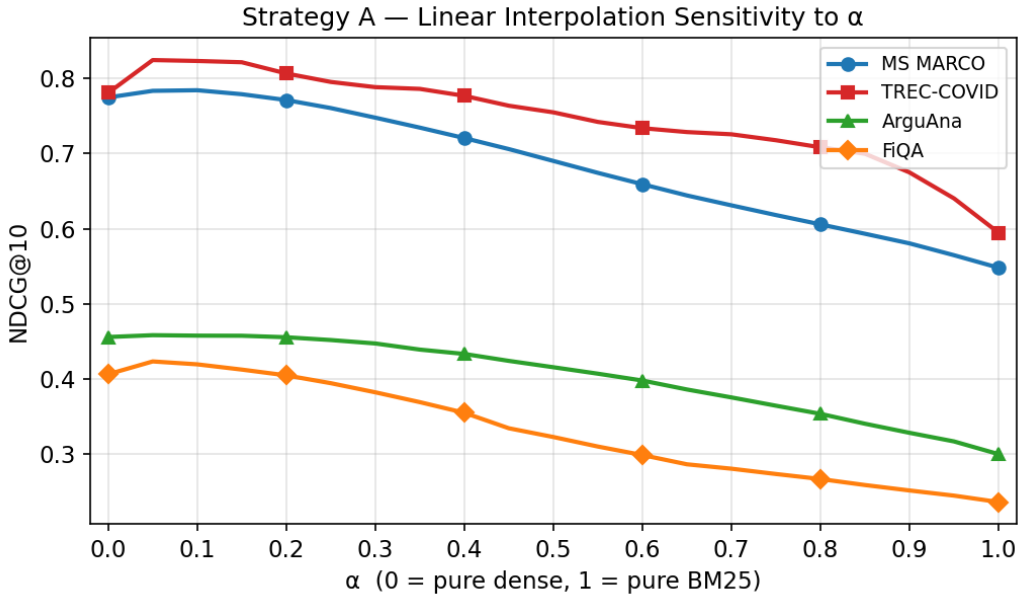

Figure 1: Strategy A alpha sensitivity. Performance peaks close to dense-only retrieval, but adding a small BM25 contribution improves NDCG@10.

The best-alpha improvements over dense-only are:

- TREC-COVID: 0.7807 to 0.8245, a gain of +0.0437 NDCG@10.
- ArguAna: 0.4558 to 0.4582, a gain of +0.0024 NDCG@10.
- FiQA: 0.4062 to 0.4233, a gain of +0.0170 NDCG@10.

RRF is less sensitive to its k parameter, but it remains below the best score-aware methods on the main metric.

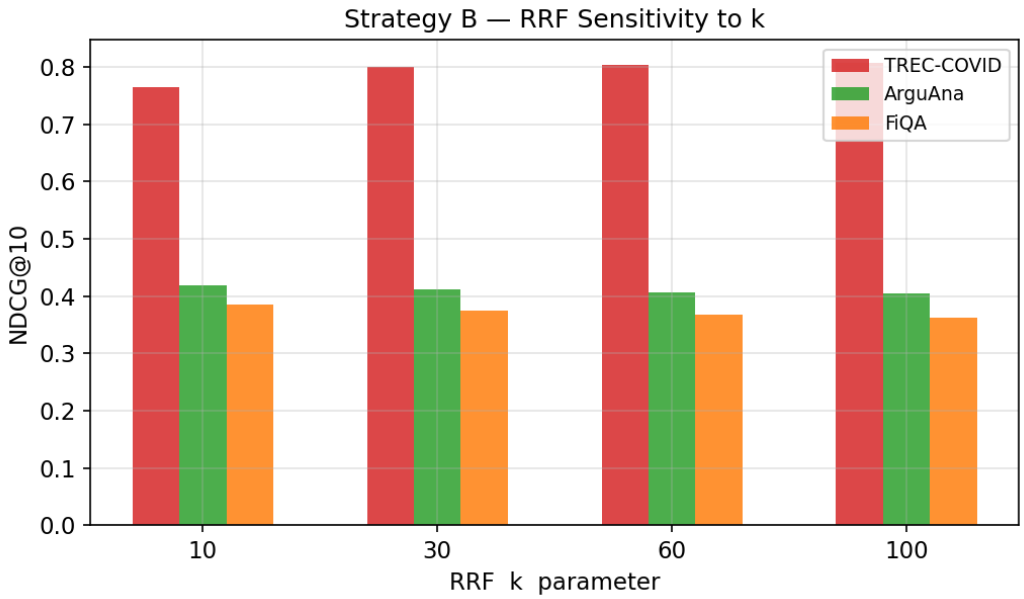

Figure 2: Strategy B RRF k sensitivity. RRF is relatively stable but does not match the best score-aware fusion methods.

5.5 Additional Query-Level Analysis

Strategy E predicts a query-specific $\hat{\alpha}$ and then uses that value in the linear fusion formula. Queries were grouped into dense-dominant, balanced, and sparse-dominant bins based on predicted $\hat{\alpha}$.

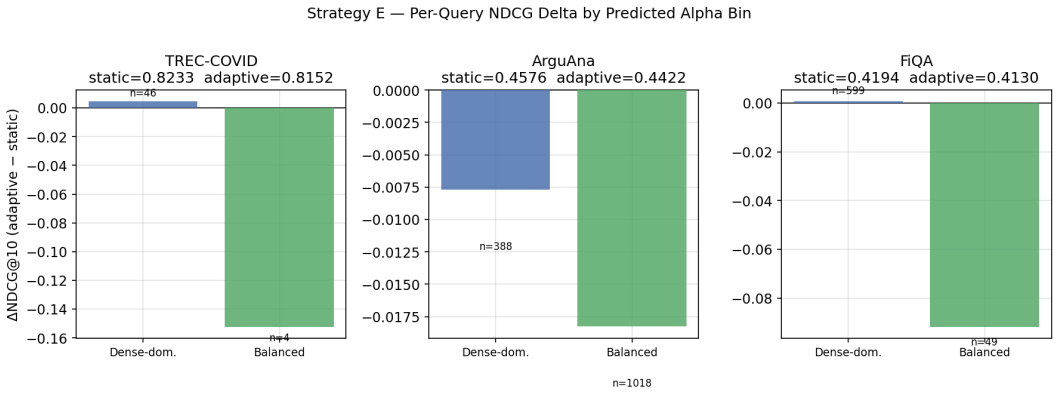

Figure 3: Query bins for Strategy E. Most queries fall into dense-dominant or moderately balanced regions, leaving relatively few clearly BM25-favoring cases.

A notable pattern in Figure 3 is that Strategy E rarely predicts high BM25 weights. The plotted query bins contain dense-dominant and balanced queries, but no sparse-dominant bin appears. For example, on TREC-COVID, 46 out of 50 queries are assigned to the dense-dominant bin, and only 4 are assigned to the balanced bin. This means that the adaptive model is not using the full range of possible interpolation weights; instead, it has learned a conservative dense-heavy policy.

The adaptive method does not consistently beat the best static α baseline. On TREC-COVID, adaptive fusion scores 0.8152 compared with static linear fusion at 0.8233. On ArguAna, it scores

0.4422 compared with 0.4576. On FiQA, it scores 0.4130 compared with 0.4194. This result suggests that the query-adaptive mechanism is plausible, but the current query features do not reliably identify when BM25 should receive substantially more weight. In other words, Strategy E is adaptive in form, but in practice, its predictions remain close to a dense-dominant fusion policy.

5.6 Oracle Upper Bound

The oracle experiment assigns each query its individually optimal interpolation weight by exhaustive grid search. For each query, we search over $\alpha \in \{0.00, 0.01, 0.02, \ldots, 1.00\}$ and select the value that maximizes NDCG@10 for that query. This step size of 0.01 gives a fine-grained estimate of the best possible linear interpolation weight under this fusion formulation.

This is not a deployable method, because it uses relevance judgments to choose the best α after the fact. Instead, it should be interpreted as an upper-bound diagnostic: it estimates the ceiling of linear interpolation if the system could perfectly choose a query-specific α.

Table 5: Oracle upper bound compared with the best achieved practical NDCG@10 in the main evaluation table.

Dataset	Best Practical NDCG@10	Oracle NDCG@10	Gap
TREC-COVID	0.8271	0.8863	+0.0592
ArguAna	0.4576	0.4987	+0.0411
FiQA	0.4194	0.4928	+0.0734

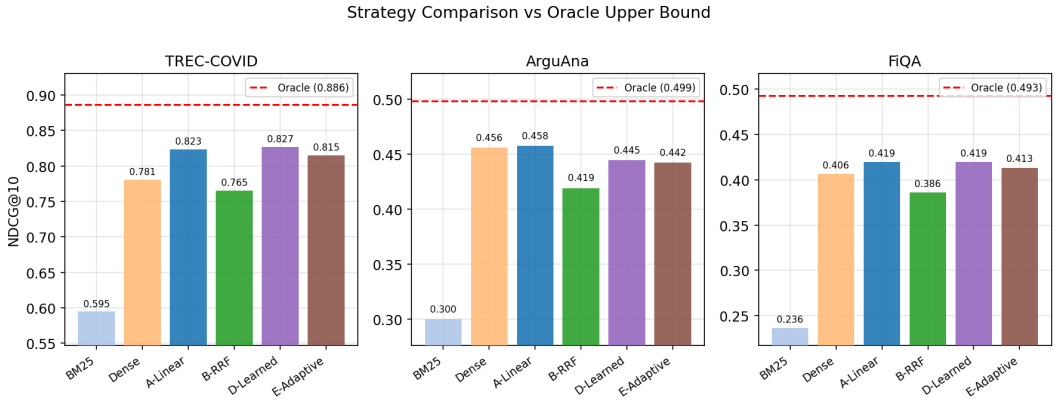

Figure 4: Oracle gap. The gap between deployable fusion and per-query oracle fusion shows that better adaptive methods could still yield meaningful gains.

The oracle results show substantial remaining headroom. This suggests that the limitation is not the linear fusion mechanism itself, but rather the difficulty of predicting the right query-specific mixing weight from simple query features.

6 Limitations and Future Work

6.1 Limitations

1. **Adaptive features are limited.** Strategy E uses a small set of query-level features. These features do not fully capture the interaction between a query and the retrieved candidate docu-

ments.

2. **Dense retrieval dominates most datasets.** The best static interpolation weights are near the dense-only end of the spectrum, which leaves limited room for adaptive fusion to improve unless it can identify rare BM25-favoring queries very accurately.

3. **MS MARCO is sub-sampled.** The full MS MARCO corpus is too large for the available compute environment, so the training/tuning corpus is reduced to 500K documents.

4. **No user-facing web interface.** The project is an experimental IR pipeline rather than a deployed search system.

5. **Limited number of dense models.** The project evaluates BGE-base-en-v1.5 as the dense retriever, but does not compare against other dense or cross-encoder models.

6. **Evaluation is benchmark-based.** The project uses standard BEIR qrels rather than live user studies or production click logs.

6.2 Future Work

1. Add stronger adaptive features such as BM25-dense rank correlation, score entropy, top-result agreement, query embedding features, or LLM-derived query type labels.

2. Replace the random forest alpha predictor with neural query encoders or learning-to-rank models trained directly on downstream ranking loss.

3. Add cross-encoder reranking after fusion to test whether hybrid fusion improves the candidate pool for more expensive rerankers.

4. Evaluate additional BEIR datasets to test whether the dense-dominant pattern holds across more domains.

5. Build a lightweight web interface that allows users to enter a query and inspect BM25, dense, and fused results side by side.

6. Improve scalability by supporting approximate FAISS indexes and reporting recall loss compared with exact dense search.

7 External Components, Prior Work, and AI Assistance

The project builds on standard open-source libraries including BEIR, Pyserini, FAISS, PyTorch, Transformers, SentenceTransformers, pytrec_eval, scikit-learn, SciPy, Matplotlib, and NumPy. These libraries provide benchmark loading, indexing, vector search, neural embedding, metric computation, machine learning models, and plotting utilities.

The submitted repository also includes an AI assistance disclosure. According to that disclosure, Claude was used for code scaffolding, boilerplate utilities, debugging assistance, and report-generation scripts. The disclosed author-owned contributions include the research questions, fusion strategy formulations, feature engineering, oracle alpha analysis, experimental design, interpretation of results, Google Colab workflow adaptation, and the MS MARCO sub-sampling strategy.

No components written for another course, prior research project, or prior employment are claimed as original work for this course

8 Conclusion

This project shows that hybrid retrieval fusion can reliably improve over either sparse or dense retrieval alone, but that not all fusion strategies are equally effective. In these experiments, simple score-aware interpolation and learned fusion are stronger than rank-only RRF. The results also show that dense retrieval dominates the evaluated datasets, with the best interpolation weights close to dense-only but still benefiting from a small BM25 contribution.

The query-adaptive fusion experiment is especially useful despite not outperforming the best static baseline. Its oracle analysis shows that large gains are theoretically possible if per-query interpolation weights could be predicted accurately. Therefore, the most promising future direction is not simply adding more fusion formulas, but designing better features or models for predicting when sparse lexical evidence should receive more weight.